NAMINGCONTEST.COM

Testing the email lifecycle without waiting days

Triggering the full email lifecycle in minutes rather than days

A real contest runs over 8–11 days. Every email it sends can be triggered in about five minutes instead, by moving the contest through the same states by hand. Nothing here is a special test mode — you are driving the real lifecycle, so what arrives is exactly what a participant would get.

Run the SQL in **Supabase → SQL Editor**.

The five emails

EMAIL	SENT BY	FIRES WHEN
Sign-in link	Supabase Auth	someone requests a magic link
Payment receipt	`confirm-launch`	Stripe confirms a payment
Your vote is needed	`notify`	contest status becomes `voting`
Time to crown the winner	`notify`	status becomes `closed` with no winner yet (creator only)
The winning name	`notify`	a winner is crowned (to everyone else)
Your name won	`notify`	a winner is crowned (to the winner)

Before you start: you need a second person

The contest emails go to **participants**, and the creator is not automatically one. Testing alone, you will send yourself nothing.

Use a second email address you control — open the invitation link in a private window, join, and submit a name. That address then receives everything a real participant would.

It is also the only way to see the product as a participant, which is worth doing once regardless.

1. Find your contest

```
select id, working_name, status, notified_voting_at, notified_winner_at
from contests
order by created_at desc
limit 10;
```

Copy the `id` you want to drive.

---

2. Trigger "Your vote is needed"

```
update contests
set status             = 'voting',
    voting_ends_at     = now() + interval '3 days',
    notified_voting_at = null
where id = 'PASTE_ID';
```

Three things happen here, and all three matter:

- `status = 'voting'` is what fires the email. The trigger watches for the *transition into* `voting`, so a contest already in that state will not re-send — see "Re-sending" below.
- `voting_ends_at` **in the future** stops the cron closing it again within the minute. It runs every 60 seconds and closes anything past its deadline.
- `notified_voting_at = null` clears the once-only stamp. Without it the function returns `{ok: true, skipped: 'already notified'}` and sends nothing — deliberately, so a status flip-flop cannot spam a real contest.

2b. Trigger "Time to crown the winner"

This one goes to the **creator**, not participants, and only fires if no winner has been picked yet.

```
update contests
set status             = 'closed',
    notified_closed_at = null
where id = 'PASTE_ID';
```

Leave `winner_submission_id` alone — setting it in the same statement means the contest closes with a winner already chosen, and the email correctly doesn't fire (you wouldn't tell someone to do what they've just done).

---

3. Trigger the winner emails

Crown a winner from the manage page — that is the honest test, since it exercises the real button. Or by hand:

```
update contests
set winner_submission_id = (
      select id from submissions
       where contest_id = 'PASTE_ID'
       order by vote_count desc
       limit 1
    ),
    notified_winner_at = null
where id = 'PASTE_ID';
```

This sends **two different emails**: the winning submitter gets "Your name won", everyone else gets "The winning name". Check both addresses — they are not the same message.

Re-sending an email you have already had

Both triggers watch for a *change*, not a state. Setting a field to the value it already holds fires nothing. To re-test, move it away and back:

```
-- Re-send the voting email
update contests set status = 'submission' where id = 'PASTE_ID';
update contests
set status = 'voting', voting_ends_at = now() + interval '3 days',
    notified_voting_at = null
where id = 'PASTE_ID';

-- Re-send the winner emails
update contests set winner_submission_id = null where id = 'PASTE_ID';
-- then crown again (step 3)
```

Two separate statements for the voting one, on purpose: the trigger fires on entering `voting`, so it has to leave first.

Letting it happen on its own

To watch the cron do the work rather than driving it yourself, launch a contest and then shorten its deadlines:

```
update contests
set submission_ends_at = now() + interval '2 minutes',
    voting_ends_at     = now() + interval '5 minutes'
where id = 'PASTE_ID';
```

Within about a minute of each deadline the cron moves the contest on and the emails go out by themselves. This is the closest thing to a real run, and the best way to check the whole sequence arrives in the right order.

When no email arrives

Work down this list — it is ordered by how often each one is the answer.

1. **Are there any participants?** The emails go to participants, not to the creator.

`sql select u.email from participants p join auth.users u on u.id = p.user_id where p.contest_id = 'PASTE_ID';`

2. **Was it suppressed as a duplicate?** `notified_voting_at`, `notified_closed_at` or `notified_winner_at` already set means the send was skipped. Null the relevant one and repeat the transition.

3. **Did the database actually call the function?**

`sql select id, status_code, left(content, 300) as response, created from net._http_response order by id desc limit 5;`

`200` means it was delivered to the function. Anything else, the response body says why.

4. **Check the spam folder**, then Resend's dashboard for the delivery record.

Resetting a test contest completely

```
delete from votes       where contest_id = 'PASTE_ID';
delete from submissions where contest_id = 'PASTE_ID';
update contests
set status               = 'submission',
    submission_ends_at   = now() + interval '3 days',
    winner_submission_id = null,
    notified_voting_at   = null,
    notified_closed_at   = null,
    notified_winner_at   = null
where id = 'PASTE_ID';
```

Participants stay joined, so you can run the whole cycle again without re-inviting anyone.

⚠️ Only against contests you are testing with. On a real one this destroys people's submissions and ballots.